

Leveraging large language models for autonomous robotic mapping and navigation

Jordán Pascual Espada¹, Sofia Yiyu Qiu¹,
Rubén González Crespo² and Juan Luis Carús³

Abstract

The ability of robots to detailedly navigate and map environments is crucial for many industrial tasks. Traditional robotic control methods, using frameworks like ROS2 (Robot Operating System) combined with tools like SLAM (Simultaneous Localization and Mapping) and NAV2 (Navigation 2), often require specialized programming knowledge, limiting accessibility for non-expert users. Recent advancements in Large Language Models (LLMs), could offer a promising alternative, enabling robots to be controlled through natural language commands. These LLM-driven systems have the potential to simplify robot interaction by translating high-level language prompts into detailed movement, mapping, and exploration tasks. There is a lack of existing solutions for utilizing natural language commands in certain movement and trajectory tasks, particularly in exploration scenarios. Current approaches are not equipped to effectively translate contextual references (e.g. next to the corner, between doors, towards the nearest unexplored area, etc.) into concrete robot commands. This proposal integrates LLMs with ROS2, SLAM, and NAV2, allowing users to issue natural language commands for detailed navigation, path planning, and mapping tasks including unexplored areas. By reducing the technical barrier to robot control, our system enables more intuitive and flexible human–robot interaction in complex environments, making robotic systems accessible to a broader range of users. In the explored scenarios, the proposed approach successfully transforms most of the users' textual prompts into a set of robot commands, achieving an F1-score of 0.913 for the successful transformation of entire prompts (composed of multiple sub-prompts) and an F1-score of 0.975 for individual commands.

Keywords

Mapping, real-time navigation, SLAM, LLMS, Chatbot, occupancy grid

Date received: 20 October 2024; accepted: 7 February 2025

Introduction

Mapping and navigation are critical tasks in the field of robotics, particularly when it comes to autonomous systems.¹ The ability to create accurate maps and efficiently move through them is essential for robots to operate in dynamic and unknown environments.² Whether it's in search and rescue missions, warehouse automation, or autonomous vehicles, the challenge lies in ensuring that the robot can understand and interact with its surroundings

¹ Department of Computer Science, University of Oviedo, Asturias, Spain

² Universidad Internacional de La Rioja, Logroño, La Rioja, Spain

³ TSK Group, Technological Scientific Park of Gijón, Gijón, Asturias, Spain

Corresponding author:

Rubén González Crespo, Department of Computer Science and Technology, Universidad Internacional de La Rioja, Logroño, Spain.

Email: ruben.gonzalez@unir.net

Data Availability Statement included at the end of the article



in real time. Traditionally, these tasks have been managed through algorithms integrated into frameworks like the robot operating system (ROS2), in combination with some representative tools like simultaneous localization and mapping (SLAM), SlamToolbox,³ and NAV2.⁴ However, programming complex movement and mapping tasks in ROS2 often requires highly specialized knowledge and is inaccessible to non-experts.

Recent advancements in large language models (LLMs), such as GPT-4, have opened new possibilities for human-robot interaction. LLMs are capable of understanding and generating human language, and their use in robotics has begun to shift how we think about programming and controlling these systems.^{5,6} Instead of relying solely on traditional programming languages, researchers are exploring ways to allow robots to be controlled through natural language commands. This reduces the technical barrier, enabling a wider range of users to interact with robots intuitively, without needing to master complex programming languages or robotic-specific frameworks like ROS2 and their tools.⁷ The ability to prompt robots using human-like dialogue can enhance flexibility in real-time decision-making, making robots more adaptive to changing environments.

Several proposals have emerged to integrate LLMs into robotic systems, focusing on translating natural language prompts into executable tasks. This involves connecting LLMs with lower-level robotic controllers that are responsible for interpreting the translated commands into actions like moving to points in the map, moving next to the objects in the map and manipulating objects.⁸⁻¹¹ However, while LLMs are excellent at language processing, they lack an understanding of the physical world and real-time constraints. Many existing proposals focus primarily on high-level movements and trajectories, often guiding robots to navigate next to objects or specific locations. However, these approaches typically do not account for scenarios that require more precise, detailed trajectories based on relative references or context elements. Additionally, they tend to overlook the importance of the map exploration process, which is critical for tasks involving unknown or dynamic environments. Existing solutions fall short when it comes to incorporating natural language commands for movement, trajectory planning, and, especially, exploration tasks. Most current methods are not designed to handle the complex process of converting references to contextual elements into specific, actionable robot commands, highlighting a significant gap in their capabilities.

In this research work, we propose a novel approach that utilizes LLMs to facilitate the control of robot mapping, explore and detailed movement tasks, based on relative information and context references. Rather than writing conventional code within ROS2 controller, users will be able to simply provide high-level instructions in natural language, which will then be translated into actionable

commands by the LLM. These commands will be processed by the robot's control system, which is integrated with ROS2 to handle the detailed execution of tasks such as detailed navigation, path planning, and map creation, features that are not common in other proposals.

Main contributions of this research work are:

- **Natural Language Interface for Robot Control:** A novel approach using LLMs to translate high-level natural language instructions into robot commands, removing the need for conventional coding in ROS2.
- **Integration of Mapping and Exploration:** This approach emphasizes the importance of map exploration and dynamic environment handling, making it suitable for unstructured or unknown environments. It also allows the use of natural language to refer to specific points on the map and to issue movement orders, enhancing usability and flexibility.

Background

In the paper CodeBotler,¹² the authors investigate how LLMs can generate robot programs based on user prompts. The system enables users to define robot functionality with concise, step-by-step instructions, which are translated into code using a high-level framework. While the robot can move to pre-identified rooms and engage in pre-defined conversations, its movement is simplified to a basic *go_to(room_name)* command. This abstraction limits the ability to handle more detailed navigation paths needed in real-world scenarios. Thus, while *CodeBotler* presents a novel use of LLMs for high-level command generation, it lacks support for precise movement control essential in advanced robotics.

ProgPrompt¹³ introduces a method for using natural language prompts to generate task plans for robots. Designed primarily for manipulation robots like robotic arms, it supports tasks such as grabbing, placing, sorting, and opening objects. The system provides guidelines for structuring prompts and incorporates AI-driven object recognition for vision-based interaction, enabling robots to understand and interact with their environment. While the approach offers valuable insights into robotic manipulation, its focus on stationary robotic arms limits its applicability to mobile robots, which require more advanced navigation and movement capabilities.

SayCan¹⁴ introduces a system that leverages LLMs to prompt a mobile manipulator robot to perform tasks, primarily focused on simulating duties typically carried out by supermarket workers, such as moving, restocking, and fetching items. The system is evaluated on tasks such as locating objects, navigating to them, picking them up, and interacting with the environment (e.g. opening drawers). Although the robot in this proposal is mobile and capable of manipulation, the approach lacks a detailed consideration of movement and path planning. The focus remains on

high-level task execution rather than the specific and precise navigation or path planning necessary for more complex robotic mobility scenarios.

The work presented in Language Planner¹⁵ investigates grounding high-level natural language tasks into executable actions, primarily in the virtual environment VirtualHome. The approach learns from step-by-step examples but struggles with mapping LLM outputs to precise actions. It focuses on high-level tasks like finding, grabbing, and placing objects, with movement commands limited to general instructions, such as moving to a named room, lacking precision for navigation or path planning. While the study evaluates how well LLMs translate prompts into actions, its emphasis on virtual environments limits applicability to real-world robotic control.

Palm-E¹¹ presents an innovative approach to combining vision and language models for embodied reasoning in robots. Integrating image and natural language inputs, enables robots to interpret multimodal instructions and perform tasks in diverse environments. While *Palm-E* improves high-level command execution, such as object recognition and navigation, it focuses on general task handling rather than precise path planning or detailed movement control. This work highlights the potential for advanced human-robot interaction but falls short in addressing fine-grained navigation and control.

The paper by Zitkovich et al.¹⁶ explores the use of LLMs to generate plans for manipulator robotic tasks in household environments. It introduces a framework that translates natural language instructions into robot-executable actions, focusing on everyday activities such as fetching objects and interacting with household items. A key feature of the system is its ability to leverage pre-trained LLMs to interpret vague or underspecified commands and convert them into detailed, structured plans. However, the paper acknowledges challenges in ensuring precise navigation and handling tasks that require real-time environmental awareness. Although this work highlights the potential of LLMs for generating complex task plans, it lacks a detailed focus on low-level path planning and movement execution, relying instead on high-level task descriptions.

The Language to Reward¹⁷ framework explores the use of natural language instructions to generate reward signals for reinforcement learning agents. This system translates language-based task descriptions into rewards that guide a robot's behavior, allowing it to perform tasks in both simulated and real environments. The approach focuses on linking high-level instructions with robot learning, enabling robots to understand tasks like object manipulation and navigation by interpreting language cues into actionable goals. While the framework excels at translating language into reward structures for learning, it does not emphasize detailed movement or path planning. Instead, its strength lies in using natural language to shape task outcomes through reinforcement learning, rather than directly controlling precise navigation or spatial awareness.

VLMaps⁹ enables robots to interpret natural language commands grounded in spatial maps from visual and language inputs. It supports high-level tasks like object recognition and navigation, including specific commands like moving to the left side of an object. The system can identify objects and execute precise movements, even specifying which side to stop on. Unlike others, *VLMaps* offers contextual object identification, such as “the chair to your east.” However, it struggles with relative movement descriptions, like “move a little,” and lacks functionality for handling unexplored areas, limiting its use in mapping and navigation.

VoxPoser¹⁸ is a framework designed to enhance human-robot interaction by allowing robots to interpret and execute natural language commands related to 3D spatial navigation and manipulation. It focuses on the task of generating detailed 3D poses and actions based on user instructions, enabling robots to interact with objects in the environment, pick up, close, sort, etc. VoxPoser can accurately interpret commands that specify not only what actions to perform but also how to position themselves in space. While VoxPoser excels in translating language into precise actions, it does not specifically address movement execution for mobile robots.

InnerMonologue¹⁹ integrates natural language understanding with cognitive reasoning to enhance robotic task execution. It enables robots to perform both manipulation and movement tasks by analyzing high-level commands like “pick up the red cup and place it on the table” or “navigate to the door and open it.” The system uses an internal dialogue to clarify ambiguous instructions and adapt to changing contexts. While *InnerMonologue* excels in high-level task reasoning, it may struggle with low-level path planning and real-time decision-making during execution.

The LMNav framework⁸ introduces an innovative method for integrating language models with robotic navigation tasks in outdoor environments, utilizing GPS and other contextual references. This framework empowers robots to interpret natural language commands and convert them into executable navigation plans for real-world scenarios. By harnessing LLMs, LMNav allows robots to comprehend intricate instructions, including multi-step tasks that involve locating and retrieving objects, while also specifying destinations, relative movements, and interactions with their surroundings. Although LMNav represents a significant advancement in the application of natural language processing for navigation, it does not adequately address the complexities of low-level path planning in indoor environments, where the structure of prompts and commands can vary significantly.

InnerMonologue²⁰ is a framework designed to enhance robotic capabilities by integrating natural language understanding with cognitive reasoning for effective task execution. This system focuses on interpreting user commands, empowering robots to perform both manipulation and movement tasks in complex environments. By employing

a multi-step reasoning process, InnerMonologue enables robots to analyze and comprehend high-level instructions, such as “pick up the red cup and place it on the table” or “navigate to the door and open it.” The framework excels in advancing robots’ abilities to understand intricate tasks, particularly those related to object recognition, retrieval, and manipulation. However, it may encounter challenges in addressing low-level navigation complexities, as many of the navigation examples provided are limited to simpler commands, such as “move next to an object,” which may not fully support dynamic navigation scenarios in more complex environments.

Another research effort that aligns closely with this work is outlined in the paper.²¹ This proposal emphasizes the ability of robots to identify and interact with objects based on natural language instructions. The focus of this research is on enhancing robotic manipulation capabilities, allowing robots to find specified objects, grasp them, and accurately place them at designated locations as indicated in the user’s prompt. The integration of advanced perception and manipulation techniques positions this research as a significant contribution to the field, particularly in contexts where precise object handling is essential. However, while the framework shows promise in terms of object manipulation and find objects.

TidyBot¹⁰ enhances robotic capabilities in household environments by enabling robots to perform cleaning and organization tasks via natural language commands. It allows users to give high-level instructions for positioning objects, such as “put the book on the shelf.” However, it may struggle with more complex navigation tasks beyond basic commands like “go to this object.” NLMap-SayCan²² follows a similar very similar objective, focus on task related to find, pick up, and go to the objects, enabling robots to interpret and execute high-level commands in complex indoor environments. Users can issue detailed instructions, such as “navigate to the kitchen and grab the blue mug,” which the robot translates into actionable tasks.

LLM-Planner²³ is a framework designed to enhance robotic task planning through the integration of LLMs with structured planning techniques. This system allows robots to interpret natural language commands and translate them into actionable plans by leveraging the contextual understanding provided by LLMs. LLM-Planner focuses on generating sequential actions that enable robots to accomplish complex tasks in dynamic environments. Navigation actions are very limited to going next to the object and finding objects.

Although generating commands from a prompt using a large language model (LLM) is not the same as producing programming code, they share important aspects such as grammar rules, instruction relationships, and the use of parameters. Numerous recent research studies have explored how to convert prompts into software applications and programming code, yielding interesting conclusions.

One such paper, STUDENTEVAL,²⁴ introduces a new benchmark for evaluating code generation models based on prompts written by non-expert programmers. This benchmark comprises 1749 prompts for 48 problems, created by 80 students who had only one semester of Python programming experience. The study found that STUDENTEVAL serves as a better discriminator of model performance compared to existing benchmarks, and it also revealed that nondeterministic LLM sampling can mislead students into believing their prompts are either more or less effective than they truly are. These findings have significant implications for how to teach with code LLMs.

In detailed indoor environments, detecting structures in the occupancy grid is crucial, providing an alternative approach to using purely visual data. Many research efforts focus on identifying various elements based on occupancy grid information, which offers robust data for solving exploration and navigation challenges. For example,²⁵ presents a novel approach for detecting door positions and completing partial occupancy maps using door-based data, introducing a door prediction algorithm for unexplored spaces. Similarly,²⁶ aims to extract the structure of an environment from a cluttered 2D occupancy grid map while performing segmentation of the map into separate rooms, a critical aspect for indoor navigation. Additionally,²⁷ demonstrates the use of occupancy grid data for detecting parked vehicles, both parallel and cross-parked, using radar information. While 2D occupancy grids are commonly employed,²⁸ explores the use of 4D grid information for detecting obstacles in the road, highlighting more advanced techniques for navigation.

Although the generation of commands from prompts is somewhat distinct from these tasks, domain-specific languages (DSLs) can offer insights. DSLs are used to specify applications for mobile robot navigation, including adaptable behavior based on external events, as explored in the work of Nordmann et al.²⁹ and in this study on DSLs for robot navigation behavior specification.³⁰ These examples highlight how the structured use of grid and language information is essential in robotic path planning and task execution.

In the analyzed works, we cannot find some critical features that facilitate user interaction with LLMs:

- **Context-Dependent Tasks:** Most tasks depend on context, such as cleaning. Very few proposals focus on generic actions, like movement, that could be applied to a wide range of scenarios.
- **Natural Language and Context Usage:** When a user describes an action related to movement or mapping, they tend to use natural language and environmental references. However, the use of context and relative references is either nonexistent or very weak in most proposals. Few proposals adequately support relative

and context-based movement commands (e.g. “move to the middle of the room,” “go to the corner”).

- **Unexplored Areas:** Practically no proposal addresses the issue of unexplored areas in maps, which is an important consideration for the types of commands a user can give to the robot.

Proposed platform

Proposal integrates three key components—natural language interface, LLM-driven command translator, and robotic execution framework—to create an intuitive system for controlling robots using natural language. The process begins with the user providing instructions via the Natural Language Interface, where their input is captured and contextualized. The interface ensures that commands are validated, checked for errors, and sent sequentially to the robot for execution. The interaction is designed to be accessible, even to users without technical expertise, as the system translates complex robotic tasks into simple commands while also providing clear feedback on the robot’s actions.

At the core of the system, the LLM-Driven Command Translator connects the user’s natural language inputs to the robotic control system. It interprets user prompts and converts them into actionable commands by leveraging a LLM, which has been fine-tuned to understand the context of robot operations. Once these commands are optimized and validated, they are sent to the Robotic Execution Framework, where they are executed by the robot using ROS2, SLAM, and NAV2 technologies. The entire architecture works together to achieve the main objective of the proposal: enabling users to control and interact with robots through natural language, simplifying the

programming process, and making robotic systems more accessible and flexible for a wider range of users and applications (Figure 1).

Architecture

Web Interface System serves as the front-end of the proposed architecture, allowing users to interact with the robot using simple, natural language inputs. Through a web-based application, the user enters commands or task descriptions in a prompt, which are then processed and forwarded to the system for further interpretation. To ensure seamless interaction, the system includes a prompt processing module that captures the user’s instructions and sends them to the LLM for translation into robotic commands. Once the language model generates a set of commands, the system validates them using a Command Validator and Plan Checker, which ensures that the commands are logically sound and free of errors related to syntax or planning. After validation, the Robot Plan Executor module sends commands to the robot sequentially and manages real-time feedback from the robot’s actions. The User Feedback Manager is responsible for taking the robot’s responses and converting them into comprehensible updates for the user, such as map data or status reports, thereby closing the interaction loop and making the robotic system accessible even to users without technical expertise.

LLM Processing Engine plays a central role in converting the user’s natural language input into executable robotic commands. This component leverages a LLM, with Google Gemini identified as the most effective model for this task

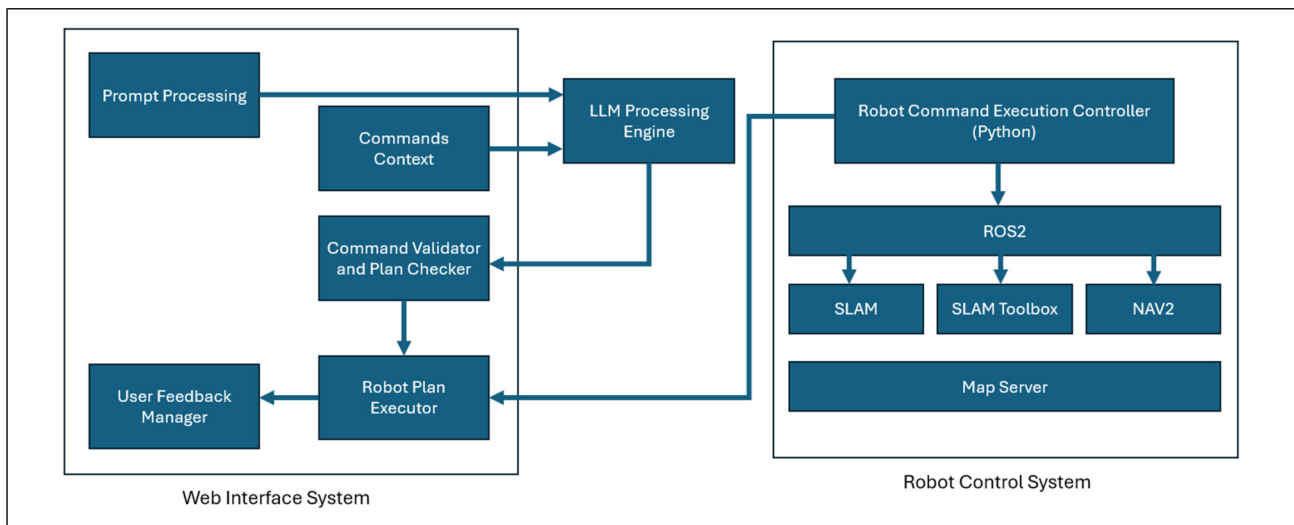


Figure 1. Proposed architecture: The “Prompt Processing” module receives the initial text and sends it to the “LLM Processing Engine.” This engine is fed by the “Commands Context” and attempts to determine the commands that should be applied to fulfill the task. The “LLM Processing Engine” sends the output commands to the “Command Validator and Plan Checker,” which applies rules to correct command errors and restructuring. A revised version of the commands is then sent to the “Robot Plan Executor,” which receives user feedback. The “Robot Command Execution Controller” requests updates from the “Robot Plan Executor.”

due to its ability to handle complex, multi-step instructions. The LLM receives the user prompt along with contextual information provided by the Command Context Generator, which outlines the robot's command syntax, parameters, and dependencies between different actions. This contributes to the LLM generating commands that the robot can understand and execute. The context ensures that even when the user's instructions are vague or incomplete, the LLM can still infer the necessary details.

Robotic execution framework (robot control system) is responsible for executing the validated commands and directly interfacing with the robot's control systems. Once the validated and optimized commands are received from the web interface, they are processed by the Robot Command Execution Controller, which converts high-level instructions into low-level actions that are compatible with ROS2, the robot's primary operating environment. The framework integrates ROS2 with SLAM (simultaneous localization and mapping) Toolbox and NAV2 for navigation, enabling the robot to perform complex tasks such as mapping an environment, navigating to specific coordinates, or avoiding obstacles. Commands are executed one at a time, with feedback from the robot being continuously monitored. The Robot Feedback Manager ensures that the robot's responses, such as positional data or completed maps, are captured and sent back to the web interface in a format that is easy for the user to understand. This modular architecture ensures real-time execution and feedback, making it possible for non-experts to effectively control robotic systems in dynamic

Prompt processing

The initial user-facing component instructions are inputted in natural language. Technically, this module is designed with a clean, intuitive interface, typically implemented as a web-based application that can handle various input types. It captures the user's text input and prepares it for further processing by breaking down the natural language prompt into structured data that the system can handle. This includes tokenizing the input and applying any necessary preprocessing, such as removing ambiguities or extracting keywords that will guide subsequent stages. The key feature of this module is its simplicity and accessibility, allowing users without technical expertise in robotics to input complex instructions with ease. The module seamlessly forwards the processed data to the LLM Connector for interpretation, initiating the process of transforming high-level natural language into actionable robot commands.

Reference elements. These are the key elements the robot can identify based on its sensor data (occupancy grid):

- Door or opening (e.g. an entry or exit point, a gap in the wall)

- Wall (e.g. a boundary, vertical surface)
- Room (e.g. a defined enclosed space)
- Hallway (e.g. a long, narrow passageway)
- Narrow passage (e.g. a tight or restricted space between obstacles)
- Obstacle (e.g. any object blocking movement)
- Corner (e.g. a meeting point of two walls)
- Unexplored area (e.g. an area that hasn't been mapped by the robot)

Element's properties. These describe additional details about the reference elements and help the robot prioritize or refine actions.

- Size: big, small, biggest, smallest
- Order/sequence: first, second, third, Nearest, farthest
- Position relative to the robot: in front, behind (back), left, right, directly ahead, to the side, adjacent, next to
- Time-based/interaction first you reach, last in the list, most recent (e.g. for obstacles), unexplored (e.g. for areas the robot hasn't seen yet)

Types of movement. The main commands that guide the robot's actions in terms of navigation

- Navigate (e.g. the robot moves to a specific location or element)
- Stop (e.g. the robot halts at a specific point)
- Move (in direction) move forward, backward, left, right (e.g. simple movements in specific directions)
- Rotate (in direction) rotate left, right (e.g. rotating the robot to face a certain direction)
- Follow (e.g. follow along a wall or path)

Movements details. These are more specific descriptions of where or how the robot should move:

- Next to (e.g. "next to the first door on your right")
- Towards (e.g. "towards the nearest unexplored area")
- Through (e.g. "through the narrow passage in front of you")
- Opposite side of (e.g. "opposite side of the biggest room")
- Along (e.g. "along the left wall of the hallway")
- Between and (e.g. "between the narrow passage and the obstacle ahead")
- Stop at (e.g. "stop at the first corner you reach")

Prompt template extraction. These main elements can extract the main information about every action of the prompt to facilitate the transformation into robot commands. Here are a few examples:

- Navigate towards the first door on your left and stop just after you pass through it.
 - Reference element: door

- Property: first, on your left
- Type of movement: navigate
- Detail about point: through
- Move to the right corner of the room you're currently in and stop there.
 - Reference element: room, corner
 - Property: right (relative to the robot's current position)
 - Type of movement: navigate, stop
 - Detail about movement point: next to the right corner
- Navigate towards the rightmost corner of the room and stop once you reach it.
 - Reference Element: Room, Corner
 - Property: rightmost
 - Type of movement: navigate, stop
 - Detail about movement point: towards the rightmost corner

Command context generator

Is a vital submodule that enhances the LLM's ability to generate commands specific to the robot's operational environment. It supplies the LLM with predefined rules and operational parameters, including the syntax of robot-compatible commands, necessary arguments, and the functional dependencies between commands. This context is dynamically created based on the current robot's state, its tasks, and environmental constraints. Technically, this component ensures that the LLM operates within a constrained space of possible outputs, narrowing down the commands to those that are executable by the robot. The Command Context Generator also plays a key role in aligning the LLM's language output with the robot's real-world capabilities, ensuring that the final command output adheres to both the robot's hardware and software limitations.

Robot commands. Robot controller is designed for processing specific commands.

- NavigateThroughPoses: This command takes a list of X and Y coordinates as parameters, instructing the robot to navigate to the specified points. The robot stops when it reaches the final coordinate. If a point is far from the robot, the NAV2 system will execute a pathfinding algorithm to determine the optimal path for reaching the target destination.
- Move: This command allows the robot to move in four directions—forward, backward, right, or left. The robot moves a limited distance of 3 cm per execution. To achieve continuous movement, this command must be issued repeatedly.
- Rotate: This command enables the robot to rotate either right or left based on the given parameter. The robot rotates 10°. To achieve continuous rotation, this command must be issued repeatedly.

- Stop: This command halts all current robot movement immediately.
- Get Current Path: This retrieves the path the robot is currently following during an active navigation session.
- Get Map: This command provides the current map generated by the robot's exploration system, returning an occupancy grid array representing the environment.
- Get Robot Pose Last: This retrieves the robot's most recent pose, including its X and Y coordinates, as well as its orientation.

Elements identification. Robot manages an occupation grid as a map, the system is not able to identify directly elements like “Door,” “Room” or “Wall” it needs to translate these concepts in a possible range of coordinates (X,Y) which could be refined by elements properties. For learning how to understand these concepts we should create some examples of what user means by these concepts and provide these examples to the LLMS as context (Figure 2).

Door or opening. A door or opening is characterized as a small gap in a wall that connects two free spaces, such as rooms or hallways. To effectively identify a door in the occupancy grid, the following steps should be performed. (1) Traverse the Grid: Iterate through the occupancy grid to examine each cell for wall segments. (2) Detect Wall Segments: Look for continuous sequences of 0 (walls) that represent either vertical or horizontal walls. (3) Identify Gaps: For each wall segment detected, check for gaps of N consecutive cells of 254 (free space) within the wall segment. The value of N max should be defined based on the expected width of doors (for instance, a value of 1 or 2 for small door openings). (4) Check Gap Borders: Ensure that the gap is bordered by 0 (walls) on both sides.

$$i = 1 \sum n \text{ grid}[x + i, y] = 0 \text{ and } \text{grid}[x + n, y] = 254, \text{ grid}[x + n + 1, y] = 0$$

Equation 1: Check Gap Borders

$$\text{grid}[x, y] = 0 \text{ and } \text{grid}[x, y + N] = 254 \text{ and } \text{grid}[x, y + N + 1] = 0$$

Equation 2: Check horizontal gaps

$$\text{grid}[x, y] = 0 \text{ and } \text{grid}[x + N, y] = 254 \text{ and } \text{grid}[x + N + 1, y] = 0$$

Equation 3: Check vertical gaps

N represents the number of consecutive free space cells indicating a gap (door).

x and y denote the coordinates in the occupancy grid where the wall segments and gaps are evaluated.

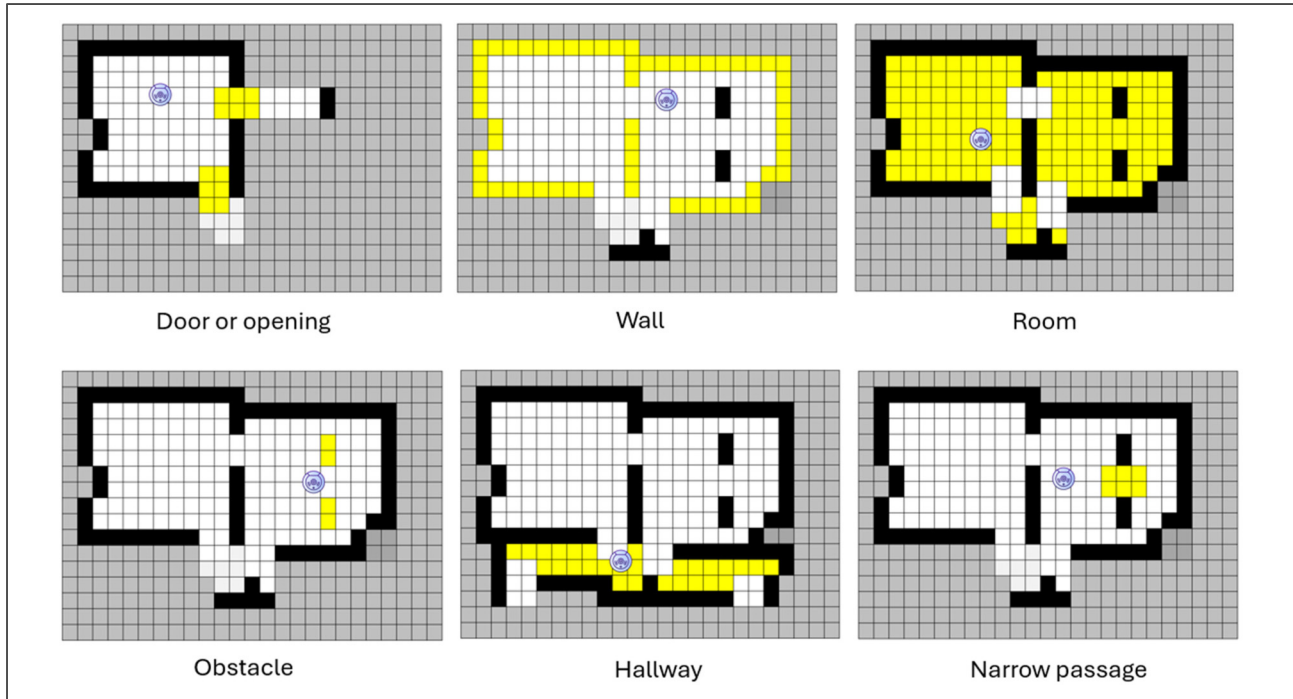


Figure 2. Some of the context elements are identified in the occupation grid. The title of the map corresponds to elements in natural language, such as “door,” “opening,” and “wall.” Black cells represent obstacles, gray cells indicate unexplored areas, and yellow cells correspond to locations referenced in the natural language nomenclature. This figure shows six of the main concepts that enable map exploration and navigation using natural language.

Walls. Walls are defined as boundaries consisting of consecutive values of 0 (walls) that enclose rooms or other spaces. In addition to detecting traditional wall segments, this algorithm also identifies obstacles, which are small areas marked by 000 that have free space (254) around them. (1) Traverse the Grid: Iterate through the occupancy grid row by row and column by column (2) Detect Continuous Wall Segments: Look for sequences of consecutive 000 values that indicate the presence of Walls (3) A wall is a continuous segment of 000 values that are connected either horizontally or vertically without interruption. Record the coordinates of all 000 values in the wall segment.

If $\text{grid}[x, y] = 0$ for n consecutive cells, then
 $\forall(x, y) \in \text{wall segment grid}[x, y] = 0$

Room. A room is an area of connected free space (254) surrounded by walls (0), which may contain openings (doors). Rooms can also contain unexplored regions (205). (1) Traverse the Grid: Iterate over each cell of the grid. For each unvisited free space cell $\text{grid}[x, y] = 254$, initiate the flood-fill process to detect a room. (2) Flood-fill is used to detect connected components of free space (254) starting from a given cell. Mark each cell as visited during the flood-fill to avoid counting it in another room detection. While filling, keep track of the boundary cells, checking if they are mostly surrounded by walls (000) or doors (small gaps in the walls). (3) During the flood-fill,

for each free space cell, check its neighboring cells (up, down, left, right, and diagonals) to identify the boundary. A cell is part of the boundary if one or more of its neighbors is a wall (000) or unexplored space (205). Doors: If there are small sequences of free space (254) within the boundary walls, detect them as possible doors. For example, a gap of 1–2 consecutive free space cells in an otherwise continuous wall could be a door. (4) Since rooms can have irregular shapes and non 90° angles, consider diagonal neighbors in the flood-fill to ensure the entire free space region is included. Ensure that the flood-fill doesn’t stop at non-right-angle boundaries, and consider free space cells as part of the same room if they are diagonally connected. (5) Allow unexplored regions (205) to be part of the room as long as they are surrounded primarily by walls (0) or free space (254). Continue the flood-fill through unexplored cells (205) if they are connected to free space but keep track of them separately, as these areas may need to be explored later. (6) Once the flood-fill is complete, store the list of all the free space cells (and possibly unexplored cells) that belong to the detected room. Add this list of cells as a new room in the output list.

$$\begin{aligned} \text{RoomFloodFill}(x0, y0) &= (x0, y0) \cup (x, y) \\ &\in \text{Neighbors}(x0, y0) \\ &\cup \text{RoomFloodFill}(x, y) \text{ if } \text{grid}[x, y] \\ &= 254 \end{aligned}$$

Algorithm 1. Room detection

Data: Occupancy grid *grid* of size *width* $\times$ *height*
Result: List of detected rooms

```

1 Function detect_rooms(grid):
2   rooms  $\leftarrow []$ ;
3   visited  $\leftarrow \emptyset$ ;
4   for  $x = 0$  to grid.width do
5     for  $y = 0$  to grid.height do
6       if grid[ $x, y$ ] = 254 and  $(x, y) \notin \textit{visited}$  then
7         room  $\leftarrow \textit{flood\_fill}(\textit{grid}, x, y, \textit{visited})$ ;
8         rooms.append(room);
9       end
10    end
11  end
12  return rooms;

```

Data: Occupancy grid *grid*, starting cell (x, y) , visited set *visited*
Result: List of cells forming a room

```

1 Function flood_fill(grid,  $x, y$ , visited):
2   stack  $\leftarrow [(x, y)]$ ;
3   room  $\leftarrow []$ ;
4   while stack is not empty do
5      $(cx, cy) \leftarrow \textit{stack.pop}()$ ;
6     if  $(cx, cy) \in \textit{visited}$  then
7       continue;
8     end
9     visited.add( $cx, cy$ );
10    room.append( $(cx, cy)$ );
11    foreach  $(nx, ny) \in \textit{get\_neighbors}(cx, cy, \textit{grid})$  do
12      if grid[ $nx, ny$ ] = 254 and  $(nx, ny) \notin \textit{visited}$  then
13        stack.append( $(nx, ny)$ );
14      end
15    end
16  end
17  return room;

```

Data: Occupancy grid *grid*, current cell (x, y)
Result: List of neighboring cells

```

1 Function get_neighbors( $x, y, \textit{grid}$ ):
2   neighbors  $\leftarrow []$ ;
3   directions  $\leftarrow$ 
4      $[(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (1, 1), (-1, 1), (1, -1)]$ ;
5   foreach  $(dx, dy) \in \textit{directions}$  do
6      $nx \leftarrow x + dx$ ;
7      $ny \leftarrow y + dy$ ;
8     if  $0 \leq nx < \textit{grid.width}$  and  $0 \leq ny < \textit{grid.height}$  then
9       neighbors.append( $(nx, ny)$ );
10    end
11  end
12  return neighbors;

```

Obstacle. This algorithm is designed to detect obstacles in a grid-based map, specifically identifying small groups of wall cells (represented by 0) that are isolated from the

main walls of the room and surrounded by free space (represented by 254). The process begins by identifying the main walls, which are the wall cells connected to the room's

Algorithm 2: Obstacle Detection in Grid Map

Input: Grid G of size $n \times m$ where $G(x, y) = 0$ for walls, $G(x, y) = 254$ for free space.

Output: List of detected obstacles

- 1 Initialize empty list *Obstacles*;
- 2 Define set *MainWalls* for connected main walls;
- 3 Define a 4-directional movement array
 $Dir = \{(0, 1), (1, 0), (0, -1), (-1, 0)\}$;
- 4 /* Step 1: Identify main walls connected to room boundary */
- 5 **foreach** *boundary cell* $(x_b, y_b) \in BoundaryofG$ **do**
- 6 **if** $G(x_b, y_b) = 0$ **then**
- 7 Perform flood fill from (x_b, y_b) to collect all connected cells with value 0;
- 8 Add connected cells to *MainWalls*;
- 9 /* Step 2: Detect isolated wall clusters (potential obstacles) */ **foreach** *cell* (x, y) **in** G **do**
- 10 **if** $G(x, y) = 0$ and $(x, y) \notin MainWalls$ **then**
- 11 Initialize empty list *Component*;
- 12 FloodFill((x, y) , *Component*);
- 13 **if** *size of Component is small (below a predefined threshold)* **then**
- 14 MarkAsObstacle(*Component*);
- 15 Add *Component* to *Obstacles*;
- 16 **Function** FloodFill((x, y) , *Component*):
- 17 Initialize stack $S = \{(x, y)\}$;
- 18 Mark (x, y) as visited;
- 19 **while** S is not empty **do**
- 20 Pop (x', y') from S ;
- 21 Add (x', y') to *Component*;
- 22 **foreach** *direction* $(dx, dy) \in Dir$ **do**
- 23 $(x_n, y_n) = (x' + dx, y' + dy)$;
- 24 **if** $G(x_n, y_n) = 0$ and (x_n, y_n) is not visited **then**
- 25 Mark (x_n, y_n) as visited;
- 26 Push (x_n, y_n) onto S ;
- 27 **Function** MarkAsObstacle(*Component*):
- 28 **foreach** $(x_o, y_o) \in Component$ **do**
- 29 Mark $G(x_o, y_o)$ as an obstacle (e.g., by assigning a new value like 128);

boundary, using a flood fill algorithm to collect all connected wall cells starting from the boundary. After identifying these main walls, the algorithm then iterates through all grid cells to

detect isolated wall clusters that are not connected to the boundary walls. For each unconnected wall cell, a flood fill is performed to gather all adjacent wall cells in a new

Algorithm 3: Corner Detection

Input: Grid of size $M \times N$ where walls are represented by 0 and free space by any other value.

Output: List of corner coordinates.

```

1 cornersList  $\leftarrow$  empty list;
2 IsCorner( $(x, y)$ ) if (IsWall( $(x, y)$ ) and (IsWall( $(x+1, y)$ ) or
   IsWall( $(x, y+1)$ ))) then
3 |   return True;
4 else
5 |   return False;
6 end
7 IsWall( $(x, y)$ ) if ( $grid[x, y] == 0$ ) then
8 |   return True;
9 else
10 |  return False;
11 end
12 for  $x \leftarrow 1$  to  $M - 1$  do
13 |   for  $y \leftarrow 1$  to  $N - 1$  do
14 | |   if IsCorner( $(x, y)$ ) then
15 | | |   AddToList( $cornersList, (x, y)$ );
16 | |   end
17 end

```

component. If the size of this component is below a certain threshold, it is classified as an obstacle and marked accordingly on the grid. This approach enables the algorithm to distinguish between large structural walls and smaller, isolated obstacles in the environment.

Narrow Passage, a narrow passage is a small corridor between two walls or obstacles, typically 1 to 2 cells wide. Search for narrow rows or columns of 254 values that are 1 to 2 cells wide. Ensure that these sequences are bordered on both sides by 000 (walls or obstacles).

$$(x, y) \in \text{boundary} \sum grid[x, y] = 254$$

Hallway, a hallway is a long and narrow path of free space (254) bordered by walls (000) on both sides but typically wider than a narrow passage. Detect sequences of 254 values that are 3 to 5 cells wide. Ensure that these sequences are bordered by 000 (walls) on both sides.

$$i = 1 \sum ngrid[x + i, y] = 254 \text{ and } grid[x + i, y + 3] = 0 \forall i$$

Corner, this algorithm identifies the corners of a room by detecting cells where horizontal and vertical walls meet at right angles, forming an “L-shape” or sharp turn. The grid represents walls as 0 and free space as any other value. The algorithm scans the entire grid, searching for grid cells that are part of the wall and have adjacent

wall cells both horizontally and vertically, indicating a corner. These corner points are then marked and stored for further use, such as navigation or mapping improvements.

Unexplored Area, Unexplored areas in a grid represent regions that the robot has not yet scanned. These areas are marked by the value 205 in the occupancy grid, and they can appear in irregular shapes, not just as individual cells or perfect squares. This algorithm identifies and groups connected unexplored cells (205) to define distinct unexplored regions. The algorithm uses a flood-fill technique to detect groups of connected cells with the value 205. Once an unexplored area is identified, it is marked and added to a list for further exploration or mapping.

Command validator and plan checker

It is designed to ensure that the commands generated by the LLM are feasible and error-free before being executed by the robot. Technically, this module implements rule-based validation systems, which check the syntax, logic, and overall feasibility of the commands against predefined operational constraints. It verifies that each command adheres to formatting requirements, ensures logical coherence between steps (especially in the case of command sequences), and checks for dependency conflicts between commands. If any errors or inconsistencies are found, the module flags them for correction or optimization. The

Algorithm 4: Unexplored Area Detection

Input: Grid of size $M \times N$ where unexplored cells are represented by 205, free space by 254, and walls by 0.

Output: List of unexplored areas, each represented as a set of cell coordinates.

```

1 unexploredAreas  $\leftarrow$  empty list;
2 IsUnexplored( $(x, y)$ ) if ( $grid[x, y] == 205$ ) then
3   | return True;
4 else
5   | return False;
6 end
7 FloodFill( $(x, y, regionSet)$ ) queue  $\leftarrow [(x, y)]$ ;
8 regionSet  $\leftarrow$  empty set;
9 while queue is not empty do
10  | current  $\leftarrow$  queue.pop();

11  | ( $cx, cy$ )  $\leftarrow$  current;
12  | if IsUnexplored( $(cx, cy)$ ) then
13  |   | regionSet.insert( $(cx, cy)$ );
14  |   | grid[ $cx, cy$ ]  $\leftarrow$  visited;
15  |   | neighbors  $\leftarrow [(cx-1, cy), (cx+1, cy), (cx, cy-1), (cx, cy+1)]$ ;
16  |   | for neighbor in neighbors do
17  |   |   | ( $nx, ny$ )  $\leftarrow$  neighbor;
18  |   |   | if ( $nx, ny$ ) is within bounds and IsUnexplored( $(nx, ny)$ )
19  |   |   |   | then
20  |   |   |   | | queue.push( $(nx, ny)$ );
21  |   |   | end
22  |   | end
23 end
24 return regionSet;
25 for  $x \leftarrow 1$  to  $M - 1$  do
26   | for  $y \leftarrow 1$  to  $N - 1$  do
27   |   | if IsUnexplored( $(x, y)$ ) then
28   |   |   | regionSet  $\leftarrow$  FloodFill( $(x, y)$ );
29   |   |   | if regionSet is not empty then
30   |   |   |   | AddToList(unexploredAreas, regionSet);
31   |   |   | end
32   |   | end
33 end

```

Command Validator also integrates a planning checker, which ensures that multi-step commands follow a logical sequence, minimizing the risk of execution errors in

complex tasks. This layer of validation is critical for preventing failures during command execution and ensuring that the robot operates efficiently and safely.

Robot plan executor

It is responsible for handling the transmission of validated commands to the robot and coordinating their sequential execution. Technically, this module establishes a communication channel between the web interface and the robot's control system, typically using APIs or middleware to ensure reliable data transfer. It manages the execution of each command one at a time, tracking the robot's progress and ensuring that feedback is received before issuing the next command in the sequence. If a command requires intermediate validation or generates a response (such as a request for clarification), the Plan Executor handles this interaction, ensuring a smooth execution flow. This module is designed to handle real-time interactions with the robot, including monitoring the execution status and pausing or adjusting command sequences if necessary. Its ability to process commands sequentially ensures that the system can adapt to real-time conditions during task execution.

Robot command execution controller

It is the core module responsible for receiving validated commands from the Web Interface System and translating them into actionable robot operations. Technically, this module integrates closely with ROS2 and its various components, such as SLAM Toolbox and NAV2, which handle tasks like mapping and navigation. The Command Execution Controller converts high-level commands into specific ROS2 actions, ensuring that each instruction is broken down into low-level operations that the robot can execute. For example, a command like "map this area" is translated into a series of ROS2-compatible instructions that involve sensor activation, data collection, and navigation planning. This module is also responsible for managing

the feedback loop between the robot and the Web Interface, ensuring that any data or responses generated by the robot are properly formatted and sent back for user review. The Command Execution Controller ensures that robotic actions are performed efficiently, with a focus on real-time responsiveness and task accuracy (Figure 3).

User feedback manager

It is the component responsible for interpreting the robot's responses and presenting them in a comprehensible format to the user. Technically, this module processes data received from the robot, which could range from simple acknowledgments (such as task completion) to more complex outputs like maps or coordinates. It translates these responses into user-friendly formats, such as visual maps or text descriptions, and displays them on the web interface. The User Feedback Manager is designed to handle a variety of data formats, ensuring that users receive clear, actionable information, whether it's the robot's current position, a status update, or detailed mapping data. This module also ensures a continuous feedback loop between the robot and the user, making it easier to track the robot's progress and make informed decisions if manual intervention is required.

Evaluation

For the proposal evaluation, we got 42 different users prompts, we did a manual extraction of the robot commands for getting the optimal results. 21 prompts are based in environment 1 and 21 prompts are based in environment 2. Environment 1 is one open room 29.69 m² Environment 2 is 6 rooms and around 206.6 m² with 7 different rooms.



Figure 3. iRobot create 3 with Raspberry Pi and AI Lidar (SLAM).

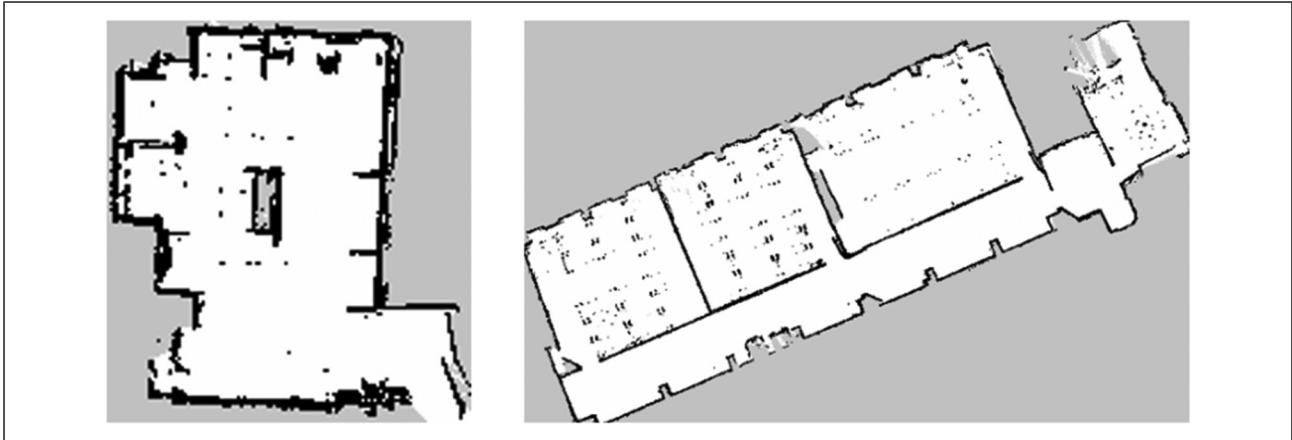


Figure 4. Environment 1 and Environment 2.

The following picture we show the environments already explored (Figure 4).

Prompts:

Move next to the door in front of you, then navigate to the middle of the room ahead and stop the movement.
 Rotate 90° to the left, move forward a little towards the nearest wall, rotate slightly to the right, then stop when you reach the end of the hallway.
 Navigate through the small door on your right and go to the far side of the room. Get the current path and stop at the nearest corner.
 Navigate forward 1 meter, move through the door ahead, rotate 90° right, and stop when you reach the unexplored area, show the current map.
 Rotate slightly left, move towards the corner in front, and stop at the narrow passage on the right, show the current map.

The Command distribution in the prompts follows the following distribution (Figures 5 and 6).

To evaluate our proposed system, we conducted a comprehensive test by inputting a series of predefined prompts into the chatbot interface, with the aim of generating commands for the robot to execute various navigation and object manipulation tasks. This evaluation focused on determining whether the system correctly translated each prompt into appropriate commands. For each prompt, we manually reviewed the generated commands to check for accuracy, examining whether the system accurately understood the task and produced the correct actions. Errors encountered during this process were categorized based on commands. These errors were quantified and are represented in the following figures, which provide a detailed breakdown of their frequency and types across different commands categories. This analysis offers insights into the system's strengths and limitations, highlighting areas where it successfully generated correct commands and areas where improvements may be needed, particularly in

handling more complex navigation tasks or specific object manipulation scenarios (Figures 7 and 8).

Finally, to obtain a comprehensive assessment of the prompt and command generation process, we calculated the recall and F1 score for all commands generated using the prompt dataset. This evaluation was conducted in two primary phases. First, we identified the prompts that resulted in all commands being generated correctly and in the proper sequence. This ensured that not only were the commands accurate, but they also followed the intended order of execution as specified in the prompts. In the second phase, we conducted a detailed evaluation of the commands generated for each individual prompt, analyzing them one by one to verify their correctness and relevance in the context of the tasks they were designed to execute.

The overall results of our evaluation are as follows: for the complete prompts, we achieved a recall score of 0.840 and an F1 score of 0.913, indicating a strong performance in capturing the intended commands. For the individual commands, the evaluation was even more promising, yielding a recall score of 0.952 and an impressive F1 score of 0.975. These metrics suggest that our system not only effectively generates accurate commands but also maintains a high level of precision and completeness across various task scenarios. This evaluation highlights the robustness of our approach in translating natural language prompts into executable commands, while also providing valuable insights into areas for further improvement and optimization in command generation accuracy and execution sequencing.

	Full prompt
Recall	0.840
F1-score	0.913
	Commands
Recall	0.952
F1-score	0.975

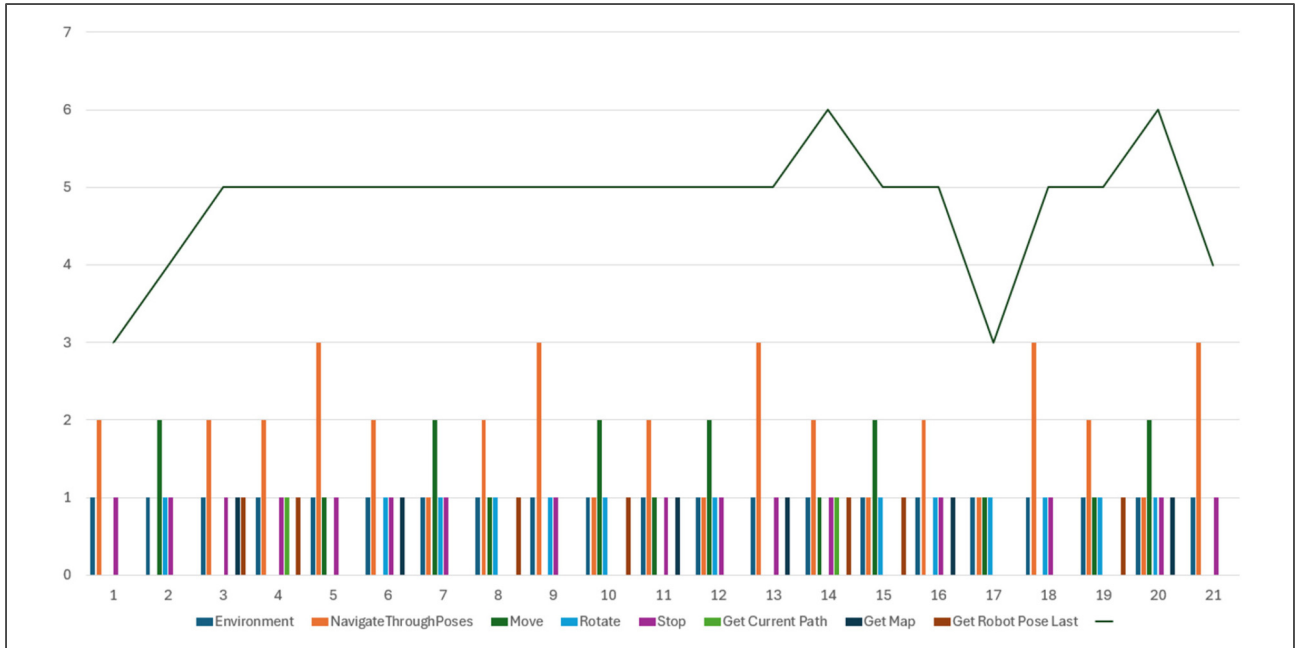


Figure 5. Command distribution in the first 21 prompts.

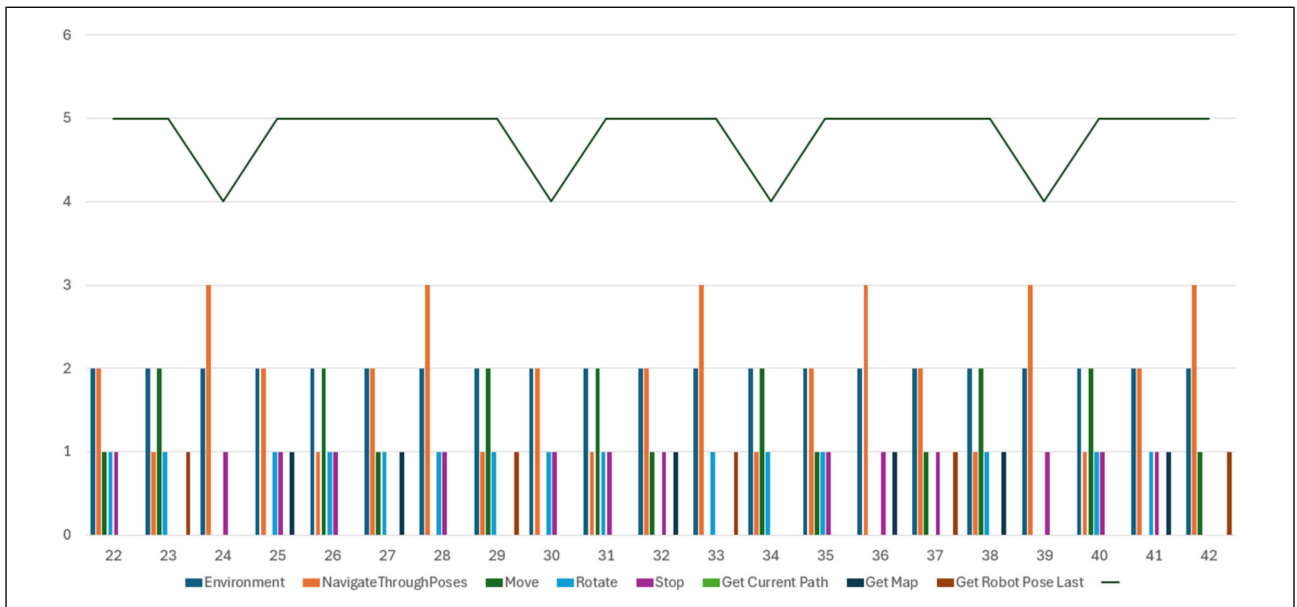


Figure 6. Command distribution in the last 21 prompts.

Most of the errors recorded during the evaluation process were associated with the generation of two specific commands: “Navigate Through Poses” and “Move.” These commands are crucial for enabling the robot to effectively traverse and interact with its environment. The recall and F1 scores for these commands are detailed in the following table:

	Navigate through poses	Move
Recall	0.941	0.974
F1-score	0.969	0.987

We aim to evaluate the quality of the x , y coordinates generated by our proposed method in relation to a robot’s

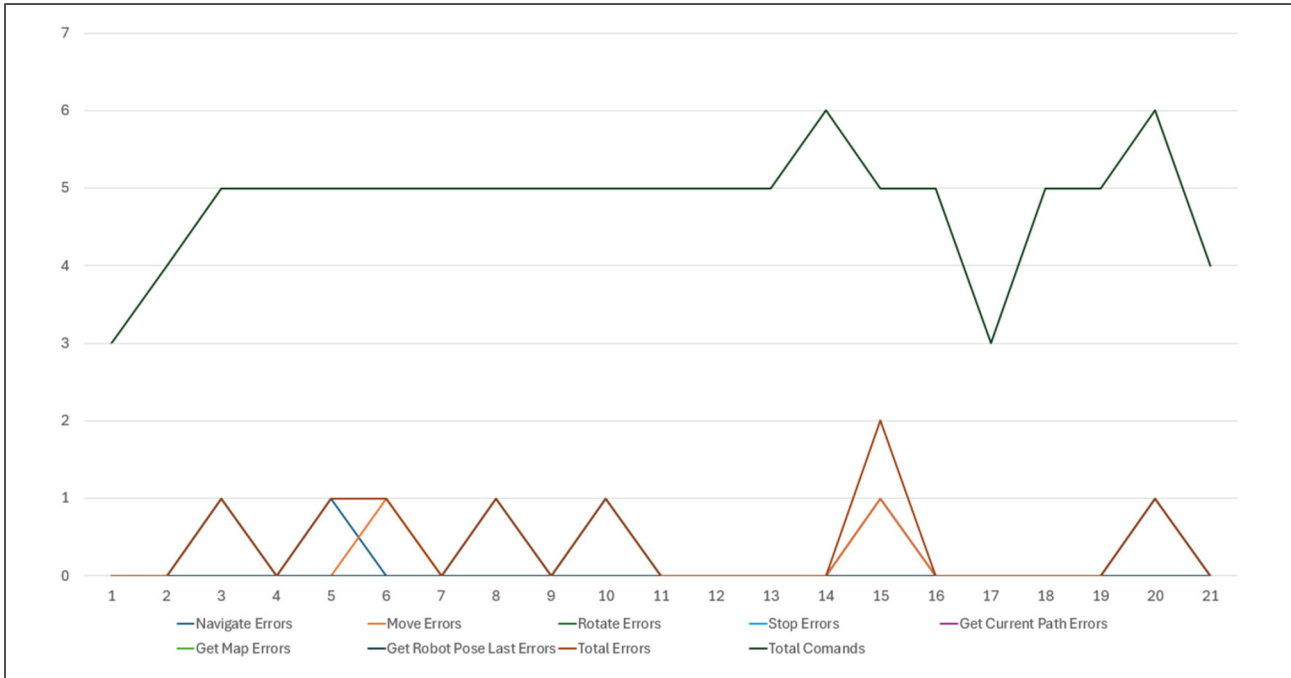


Figure 7. Errors distribution in the command type generation of the first 21 prompts.

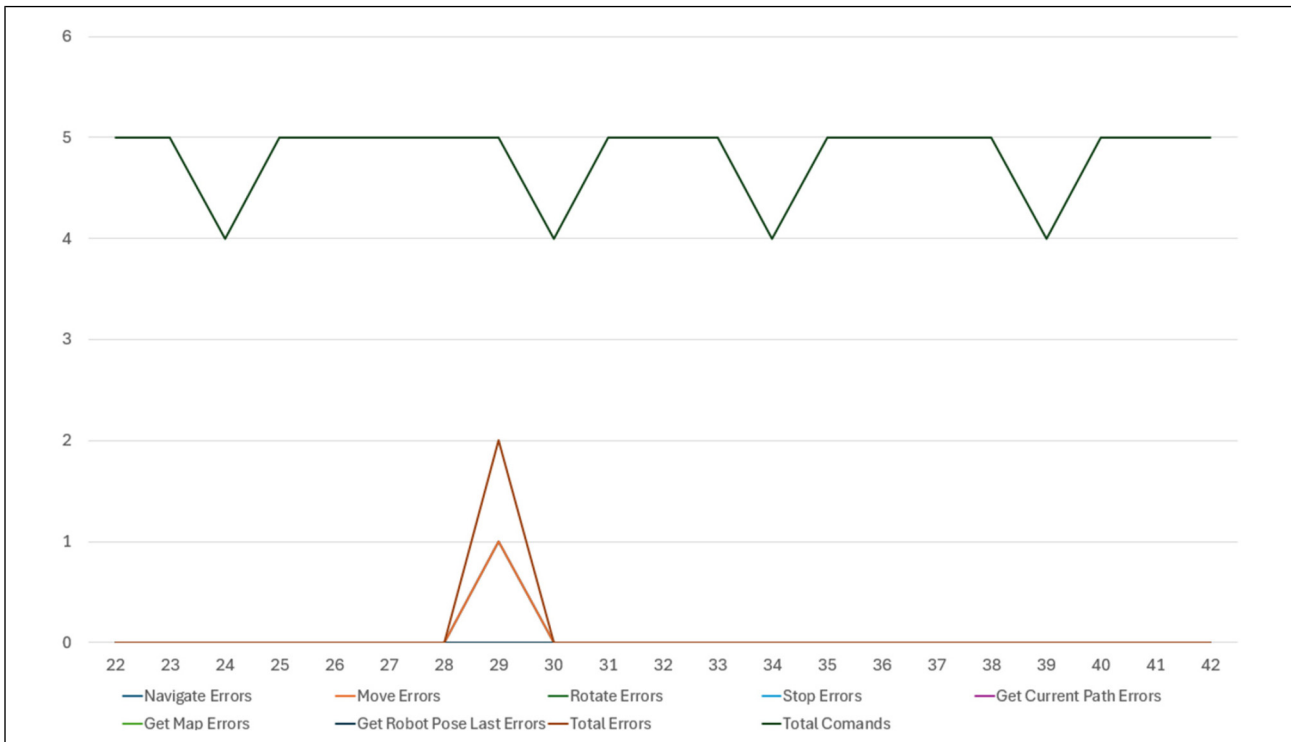


Figure 8. Errors distribution in the command type generation of the last 21 prompts.

navigation tasks. Specifically, we focus on two key commands: “Navigate Through Poses” and “Move,” both of which utilize parameters such as x , y for movement direction and rotation. To assess the accuracy of the generated

coordinates, we compare them against a predefined set of valid positions represented by a 2D array of potential successful coordinates. This array is derived from the robot’s occupancy grid map, which indicates safe and feasible

locations for navigation within the environment. The evaluation results for the two commands are summarized below:

	NavigateThroughPoses	Move
Recall	0.784	0.926
F1-score	0.879	0.962

The recall score for the “Navigate Through Poses” command is 0.784, suggesting that while a significant number of relevant positions were successfully identified, there remains a considerable proportion of potential navigation points that were not captured. This indicates a need for improvement in generating coordinates that align more closely with valid poses in the robot’s operational environment. In contrast, the “Move” command exhibits a much higher recall of 0.926, reflecting a robust ability to identify valid movement coordinates. The corresponding F1 scores further illustrate this performance, with “Navigate Through Poses” achieving an F1 score of 0.879 and “Move” reaching 0.962. These scores indicate a strong balance between precision and recall for the “Move” command, suggesting that it is not only effective at identifying correct movement coordinates but also minimizes false positives. The evaluation results highlight the robustness of our system in generating accurate commands from natural language prompts, with high recall and F1 scores for both complete prompts (0.840 and 0.913) and individual commands (0.952 and 0.975). However, the “Navigate Through Poses” command showed lower performance (recall 0.784, F1 0.879), indicating areas for improvement in generating valid navigation coordinates. In contrast, the “Move” command demonstrated strong performance (recall 0.926, F1 0.962). Focusing on enhancing the “Navigate Through Poses” command will further improve the system’s reliability and effectiveness in robotic navigation tasks.

Conclusions and future work

In conclusion, this research highlights the significant potential of integrating LLMs to enhance robotic detailed navigation and mapping capabilities. By leveraging natural language commands, our system simplifies user interaction, making it accessible to individuals without specialized programming knowledge. The successful evaluation of our approach demonstrates that users can now perform detailed movements and explorations by providing high-level textual instructions. The positive evaluation results, including recall and F1 scores, indicate that the system can effectively generate valid navigation commands. The evaluation metrics obtained during testing further underscore the efficacy of our proposed solution. While the “Move” command exhibited high recall and F1 scores, the “Navigate Through Poses” command revealed areas for improvement, particularly in the identification of relevant navigation points.

Looking ahead, future work will focus on expanding the system’s capabilities by enabling the identification of additional physical elements within indoor environments, such as windows, tables, and other objects. By incorporating these features, we can enhance the context and relevance of the commands issued to the robot, allowing for even more sophisticated navigation and interaction. The ability to reference specific objects will not only improve the precision of the generated commands but will also enrich the overall user experience. As we continue to develop this system, we aim to further empower users to engage with robotic systems in their daily tasks, ultimately leading to broader applications and advancements in the field of autonomous robotics.

Declaration of conflicting interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The authors received no financial support for the research, authorship, and/or publication of this article: Financial Support: Immersive Physical Security System Based on Autonomous and Teleoperated Robotic Platforms. R&D Projects Program 2022 (Agency, Agency for Science, Business Competitiveness, and Innovation of AsturiasSpain) IDE/2022/000605.

ORCID iD

Rubén González Crespo  <https://orcid.org/0000-0001-5541-6319>

Data availability statement

References

1. Habib MK. Real time mapping and dynamic navigation for Mobile robots. *Int J Adv Robot Syst* 2007; 4: 35.
2. Huang GQ, Rad AB and Wong YK. A new solution to map dynamic indoor environments. *Int J Adv Robot Syst* 2006; 3: 30.
3. Macenski S and Jambrecic I. SLAM Toolbox: SLAM for the dynamic world. *J Open Source Softw* 2021; 6: 2783.
4. Macenski S, Martin F, White R, et al. The marathon 2: a navigation system. In: CoRR. 2020, p. abs/2003.00368.
5. Zhang C, Chen J, Li J, et al. Large language models for human-robot interaction: a review. *Biomimetic Intell Robot* 2023; 3: 100131.
6. Mandi Z, Jain S and Roco SS. Dialectic multi-robot collaboration with large language models. In: 2024 IEEE International Conference on Robotics and Automation (ICRA) 2024, pp.286–299.
7. Huang S and Dissanayake G. A critique of current developments in simultaneous localization and mapping. *Int J Adv Robot Syst* 2016; 13: 1729881416669482.

8. Shah D, Osinski B, Ichter B, et al. LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action 2022.
9. Huang C, Mees O, Zeng A, et al. Visual Language Maps for Robot Navigation 2023.
10. Wu J, Antonova R, Kan A, et al. Tidybot: personalized robot assistance with large language models. *Auton Robots* 2023; 47: 1087–1102.
11. Driess D, Xia F, Sajjadi MSM, et al. PaLM-E: An Embodied Multimodal Language Model 2023.
12. Hu Z, Lucchetti F, Schlesinger C, et al. Deploying and evaluating llms to program service mobile robots. *IEEE Robot Autom Lett* 2024; 9: 2853–2860.
13. Singh I, Blukis V, Mousavian A, et al. Progprompt: generating situated robot task plans using large language models. In: 2023 IEEE International Conference on Robotics and Automation (ICRA) 2023: 11523–11530. <https://doi.org/10.1109/ICRA48891.2023.10161317>
14. Ahn M, Brohan A, Brown N, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances 2022.
15. Huang W, Abbeel P, Pathak D, et al. Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents 2022.
16. Brohan A, Brown N, Carbajal J, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. 2023
17. Yu W, Gileadi N, Fu C, et al. Language to Rewards for Robotic Skill Synthesis 2023.
18. Huang W, Wang C, Zhang R, et al. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models 2023.
19. Huang W, Xia F, Xiao T, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models 2022.
20. Huang W, Xia F, Xiao T, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models 2022.
21. Ding Y, Zhang X, Paxton C, et al. Task and Motion Planning with Large Language Models for Object Rearrangement 2023.
22. Chen B, Xia F, Ichter B, et al. Open-vocabulary Queryable Scene Representations for Real World Planning 2022.
23. Song CH, Wu J, Washington C, et al. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models 2023.
24. Babe HM, Nguyen S, Zi Y, et al. StudentEval: A Benchmark of Student-Written Prompts for Large Language Models of Code 2023.
25. Güzel S, Uslu E and Yavuz S. Occupancy grid map completion based on geometric features and door discovery. In: 2024 12th International Symposium on Digital Forensics and Security (ISDFS) 2024, pp. 1–6. <https://doi.org/10.1109/ISDFS60797.2024.10527317>.
26. Luperto M, Kucner TP, Tassi A, et al. Robust structure identification and room segmentation of cluttered indoor environments from occupancy grid maps. *IEEE Robot Autom Lett* 2022; 7: 7974–7981.
27. Dubé R, Hahn M, Schütz M, et al. Detection of parked vehicles from a radar based occupancy grid. In: 2014 IEEE Intelligent Vehicles Symposium Proceedings 2014, pp.1415–1420. <https://doi.org/10.1109/IVS.2014.6856568>
28. Richter E, Lindner P, Wanielik G, et al. Advanced occupancy grid techniques for lidar based object detection and tracking. In: 2009 12th International IEEE Conference on Intelligent Transportation Systems 2009, pp. 1–5. <https://doi.org/10.1109/ITSC.2009.5309730>.
29. Nordmann A, Hochgeschwender N and Wrede S. A survey on domain-specific languages in robotics. In: Davide B and Broenink JF, KT and MBA (eds) *Simulation, modeling, and programming for autonomous robots*. Cham: Springer International Publishing, 2014, pp. 195–206.
30. Santos AC, Cardoso JMP, Diniz PC, et al. Specifying adaptations through a DSL with an application to mobile robot navigation. *OpenAccess Series in Informatics* 2013; 29: 219–234.